Manual de usuario

1. Objetivo del sistema

Este sistema permite gestionar productos de supermercado y, al mismo tiempo, comparar el comportamiento de varias estructuras de datos en operaciones de insercion, busqueda, eliminacion, visualizacion y exportacion.

El programa trabaja con cinco estructuras principales:

- Arbol AVL (indice por nombre)
- Arbol B (indice por fecha de vencimiento)
- Arbol B Plus (indice por categoria)
- Lista enlazada
- Lista ordenada

Cada producto se maneja como una entidad unica que contiene:

- Nombre
- Codigo de barras
- Categoria
- Fecha de vencimiento
- Marca
- Precio
- Stock

2. Requisitos previos

Antes de usar el sistema, asegurese de tener:

- CMake instalado
- Un compilador compatible con C++17
- Terminal de Windows (PowerShell o CMD)
- Graphviz (opcional, para convertir .dot a imagen)

3. Estructura basica del proyecto

En la raiz del proyecto encontrara, entre otros, estos elementos:

- CMakeLists.txt
- source
- build

- output
- productos_ejemplo.csv
- productos_1000.csv

4. Compilacion paso a paso en Windows

4.1. Configurar el proyecto

Ejecute desde la carpeta raiz:

cmake -S . -B build

- Salida esperada aproximada:
- The CXX compiler identification is ...
- Detecting CXX compiler ABI info
- Configuring done
- Generating done
- Build files have been written to: ...\build

4.2. Compilar

cmake --build build --config Release

Salida esperada aproximada:

[10%] Building CXX object ...

[80%] Linking CXX static library ...

[100%] Linking CXX executable supermercado_app.exe

4.3. Ejecutar

Dependiendo del generador de CMake, el ejecutable puede quedar en una de estas rutas:

- build\Release\supermercado_app.exe
- build\supermercado_app.exe

Ejemplo de ejecucion:

build\Release\supermercado_app.exe

5. Pantalla principal del programa

Al iniciar, el programa muestra el menu principal:

==========

SISTEMA DE GESTION - SUPERMERCADO

==========

1. Insertar Producto

2. Insertar desde CSV

3. Eliminar Producto

4. Buscar Producto

5. Mostrar Listas

6. Exportar Arboles a .DOT

7. Medir Latencias de Querys

8. Salir

Opcion (1-8):

6. Opcion 1: Insertar producto manualmente

Esta opcion crea un producto y lo inserta en todas las estructuras.

6.1. Flujo de entrada

El sistema solicitara:

- Nombre del producto
- Codigo de barras
- Categoria
- Fecha de vencimiento (YYYY-MM-DD)
- Marca
- Precio
- Stock

6.2. Ejemplo en terminal

Opcion (1-8): 1

==========

INSERTAR NUEVO PRODUCTO

==========

Nombre del producto: arroz premium

Codigo de barras: 7701234567890

Categoria: granos

Fecha de vencimiento (YYYY-MM-DD): 2027-10-30

Marca: Andina

Precio: 18.9

Stock: 55

[Success] Producto insertado en todas las estructuras de datos.

6.3. Errores comunes

- Nombre vacio
- Fecha con formato incorrecto
- Codigo o clave duplicada en alguna estructura

Ejemplo de error:

Fecha de vencimiento (YYYY-MM-DD): 30-10-2027

Fecha invalida. Usa formato YYYY-MM-DD (ej: 2026-03-15):

7. Opcion 2: Insertar desde CSV

Esta opcion carga muchos productos de una sola vez y muestra estadisticas de carga.

7.1. Formato esperado del CSV

El archivo debe tener columnas en este orden:

Nombre, Codigo de barras, Categoria, Fecha de vencimiento, Marca, Precio, Stock

Ejemplo de una fila valida:

"Leche entera","750100000001","Lacteos","2026-12-01","Marca A",18.50,12

7.2. Ejecucion de ejemplo

Opcion (1-8): 2

==========

INSERTAR DESDE ARCHIVO CSV

==========

Ruta del archivo CSV: productos_1000.csv

Cargando archivo...

=== ESTADISTICAS DE CARGA ===

Total de lineas: 1000

Productos cargados: 998

Errores totales: 2

=== INSERCION EN ESTRUCTURAS ===

Arbol AVL - 41 milisegundos

Arbol B - 28 milisegundos

Lista Enlazada - 65 milisegundos

Lista Ordenada - 92 milisegundos

Arbol B Plus - 36 milisegundos

Total de productos: 998

Revisa error.log para detalles de errores.

[Success] Carga desde CSV completada.

7.3. Archivo de errores

Si el CSV contiene problemas, se registran en error.log. Ejemplos tipicos:

- Fecha invalida
- Columna faltante
- Precio no numerico
- Codigo de barras repetido

8. Opcion 3: Eliminar producto

Permite eliminar por nombre o por fecha de vencimiento.

8.1. Flujo de seleccion

Por cual clave deseas eliminar?

1. Por Nombre

2. Por Fecha de Vencimiento

3. Cancelar

8.2. Ejemplo de eliminacion por nombre

Opcion (1-8): 3

Por cual clave deseas eliminar?

1. Por Nombre

2. Por Fecha de Vencimiento

3. Cancelar

Opcion (1-3): 1

Nombre del producto a eliminar: arroz premium

Eliminando producto de todas las estructuras...

[OK] Eliminado de Arbol AVL

[OK] Eliminado de Arbol B

[OK] Eliminado de Arbol B Plus

[OK] Eliminado de Lista Enlazada

[OK] Eliminado de Lista Ordenada

[Success] Producto eliminado de todas las estructuras.

8.3. Comportamiento de seguridad

Si falla la eliminacion en alguna estructura, el sistema intenta rollback para restaurar consistencia.

9. Opcion 4: Buscar producto

Permite escoger la estructura sobre la que se consulta.

9.1. Menu de busqueda

En cual estructura deseas buscar?

1. Arbol AVL (por Nombre)
2. Arbol B (por Fecha de Vencimiento)
3. Arbol B Plus (por Categoria)
4. Lista Enlazada (por Nombre)
5. Lista Ordenada (por Nombre)
6. Cancelar

9.2. Ejemplo de busqueda en AVL

Opcion (1-6): 1

Nombre a buscar: arroz premium

=== PRODUCTO ENCONTRADO ===

Nombre: arroz premium

Codigo: 7701234567890

Categoria: granos

Marca: Andina

Precio: $18.90

Stock: 55

Vencimiento: 2027-10-30

[Success] Busqueda completada.

9.3. Ejemplo de busqueda sin resultado

Opcion (1-6): 4

Nombre a buscar: producto_que_no_existe

[Fallo] ERROR: Producto no encontrado en Lista.

10. Opcion 5: Mostrar listas

Muestra productos en lista enlazada o lista ordenada.

10.1. Ejemplo de salida

==========

LISTA ORDENADA

==========

Lista ordenada (5 productos):

- aceite vegetal
- arroz premium
- azucar morena
- leche entera
- pasta integral

11. Opcion 6: Exportar arboles a DOT

Genera archivos .dot para AVL, B y B Plus.

11.1. Flujo de exportacion

Que arbol deseas exportar?

1. Arbol AVL
2. Arbol B
3. Arbol B Plus
4. Todos
5. Cancelar

Directorio de salida (ej: output) [Enter=current]: output

Generando archivos DOT...

Generando Arbol AVL... [OK]

    Archivo: output/arbol_avl.dot

Generando Arbol B... [OK]

    Archivo: output/arbol_b.dot

Generando Arbol B Plus... [OK]

    Archivo: output/arbol_bplus.dot

[Success] Exportacion completada.

11.2. Conversion de DOT a PNG con Graphviz

Ejemplos de comandos:

```
dot -Tpng output\arbol_avl.dot -o output\arbol_avl.png
dot -Tpng output\arbol_b.dot -o output\arbol_b.png
dot -Tpng output\arbol_bplus.dot -o output\arbol_bplus.png
```

11.3. Verificacion rapida

Despues de ejecutar los comandos, verifique que existan estos archivos:

- output\arbol_avl.png
- output\arbol_b.png
- output\arbol_bplus.png

12. Opcion 7: Medir latencias de querys

Esta opcion mide tiempo promedio de busqueda por nombre en:

- Lista enlazada
- Lista ordenada
- Arbol AVL

12.1. Datos solicitados por el sistema

- Cantidad de consultas por corrida
- Cantidad de repeticiones
- Nombres existentes para el caso de exito

12.2. Ejemplo de ejecucion

Opcion (1-8): 7

==========

MEDICION DE LATENCIAS DE QUERYS (NOMBRE)

==========

Cantidad de consultas por corrida (ej: 20): 5

Cantidad de repeticiones (ej: 5): 10

Ingresa 5 nombres existentes para pruebas exitosas:

Nombre existente #1: arroz premium

Nombre existente #2: leche entera

Nombre existente #3: pasta integral

Nombre existente #4: azucar morena

Nombre existente #5: aceite vegetal

=== RESULTADOS (promedio por consulta en microsegundos) ===

Caso EXITO:

- Lista Enlazada : 12.45 us
- Lista Ordenada : 8.17 us

- Arbol AVL : 2.91 us

Caso FALLO:

- Lista Enlazada : 14.62 us
- Lista Ordenada : 9.50 us
- Arbol AVL : 3.10 us

N consultas: 5 | M repeticiones: 10

[Success] Benchmark de latencias completado.

12.3. Recomendaciones para medir bien

- Use un dataset grande, por ejemplo productos_1000.csv.
- Repita la prueba al menos 3 veces.
- Mantenga el equipo sin tareas pesadas en segundo plano.
- Use la misma lista de consultas para comparar corridas.

13. Opcion 8: Salir

Termina el programa de forma controlada.

Ejemplo:

Opcion (1-8): 8

Gracias por usar el sistema de gestion!

14. Reglas de validacion aplicadas por el sistema

- El nombre no puede ser vacio.
- El codigo de barras debe ser unico en la carga CSV.
- La fecha debe respetar YYYY-MM-DD.
- El precio debe ser decimal valido.
- El stock debe ser entero valido.

15. Guia de solucion de problemas

15.1. El programa no ejecuta

Verifique:

- Que la compilacion haya finalizado sin errores.
- Que este usando la ruta correcta del ejecutable.
- Que tenga permisos sobre la carpeta del proyecto.

15.2. Error al cargar CSV

Verifique:

- Ruta correcta del archivo.
- Separador por coma.
- Columnas completas.
- Formato de fecha.

Revise tambien error.log para diagnostico puntual.

15.3. No se generan imagenes PNG

Verifique:

- Que Graphviz este instalado.

- Que el comando dot este disponible en terminal.
- Que los archivos .dot existan en output.

16. Flujo recomendado para una demostracion completa

1. Compilar y ejecutar el programa.
2. Cargar productos_ejemplo.csv para validar flujo.
3. Insertar manualmente uno o dos productos.
4. Buscar productos en varias estructuras.
5. Eliminar un producto y validar que desaparece.
6. Cargar productos_1000.csv para prueba de volumen.
7. Medir latencias con varias repeticiones.
8. Exportar arboles a DOT y convertir a PNG.
9. Incorporar resultados y capturas al reporte academico.

17. Buenas practicas de uso

- Mantenga nombres consistentes en minusculas para busquedas por nombre.
- Evite duplicar informacion en inserciones manuales.
- Genere respaldos de CSV antes de pruebas grandes.
- Guarde evidencias de tiempos en una tabla para el reporte.